MedGov-AI: An MCP-Based Agentic Orchestrator for Clinical Medical Image Analysis and Structured Report Generation

***Abstract*—**

***Index Terms*—Agentic AI, Medical Imaging, MCP, MONAI, Clinical Workflow Automation**

I. Background

A. AI in Clinical Practice

Healthcare systems worldwide face increasing demand with limited resources. The volume of medical imaging studies continues to grow while the number of specialists remains constrained, creating diagnostic backlogs that directly affect patient outcomes. Automating routine analysis tasks is therefore not merely a convenience but a structural necessity. Greater automation brings greater consistency, reduces the workload on healthcare professionals, accelerates diagnosis, and allows clinicians to dedicate more time to complex cases and direct patient care. Two questions frame the motivation for this work:

What AI tools are deployed in clinical settings and what do they do? Several AI tools are currently used across clinical disciplines, with the notable characteristic that they typically operate in isolation from one another, with no unified integration layer. In radiology, AI assists with worklist prioritization and critical finding alerts, where the system detects life-threatening conditions such as stroke, pulmonary embolism, or intracranial hemorrhage and immediately notifies the clinical team. Mass General Brigham (United States) has deployed AI-driven radiology worklist prioritization to reduce time-to-read for urgent studies. The NHS (United Kingdom) deployed Annalise.ai across more than 40 hospital trusts for chest X-ray triage, allowing radiologists to focus on flagged abnormal cases first. In screening programs, AI acts as a first filter, flagging high-risk cases for specialist review and allowing programs to scale without proportionally increasing the number of specialists required. Google Health conducted a published clinical trial of an AI system for diabetic retinopathy screening in Thailand, demonstrating that AI-assisted grading matched or exceeded specialist performance in a real clinical setting []. Mayo Clinic (United States) has integrated AI into ECG interpretation and cardiac imaging workflows as a decision support layer for cardiologists. In pathology, whole slide image analysis is used for tumour grading and cell counting, and AI-generated preliminary reports from imaging findings are now in active clinical use. For segmentation and detection specifically, FDA-cleared tools such as Viz.ai and Aidoc are deployed for CT-based triage; Transpara and iCAD for mammography screening; and IDx-DR for diabetic retinopathy grading from fundus images.

What are the benefits and what are the risks? The benefits of AI in clinical workflows are directly tied to the limitations of the current manual process. Faster analysis is the most immediate gain: AI systems can process and flag findings in seconds, reducing the time between acquisition and clinical decision. Consistency is a second key advantage, as AI models apply the same criteria uniformly across every case, unlike human specialists who are subject to fatigue, experience, and workload. Reducing routine reads and preliminary reporting also frees specialists for cases that genuinely require expert judgement. Taken together, these properties position AI not as a replacement for clinicians but as a force multiplier. The risks are equally significant. Bias is a primary concern: models trained on data from specific demographics or equipment may underperform on underrepresented groups or institutions using different hardware. Hallucination is critical in language models used for report generation, where confident but clinically incorrect statements carry direct patient risk. The black-box nature of deep learning raises accountability concerns, as clinicians and regulators often cannot explain why a model reached a particular conclusion. From a regulatory standpoint, no unified framework governs how AI tools in healthcare should be validated, updated, or audited in production. Finally, and most relevant to this work, no established standards govern how agentic AI systems should operate in clinical environments, including how they communicate with existing tools, when they escalate to a human, and how their decisions are logged for audit.

Despite the progress of individual AI tools, no standard integration layer exists to orchestrate them together within a clinical workflow. Each institution must develop its own connectors and its own standards, which leads to fragmentation, lack of interoperability, slow adoption, and deployments that are expensive and difficult to maintain. The Model Context Protocol (MCP)[1] is a promising solution to this problem, providing a standardized way for agents to discover and call tools without bespoke integration. This work explores the application of MCP as the integration layer for an agentic

[1]MCP is an open protocol that defines a standardized client-server interface through which an AI agent discovers and invokes external tools. Each MCP server declares its available tools at startup with names, input schemas, and natural language descriptions; the agent selects and calls them by name at runtime without any prior knowledge of the server's implementation.

AI system that orchestrates multiple clinical imaging tools in a real-world workflow.

B. Agentic AI in Healthcare

Agentic AI[2] refers to systems that can autonomously set goals, plan multi-step actions, and use tools to achieve those goals. Unlike traditional AI models that operate in a single step or require human input at every stage, agentic systems can reason through complex tasks, make decisions about which tools to use, and adapt their behavior based on feedback. This makes them particularly well-suited for clinical imaging workflows, which are inherently sequential and often require the integration of multiple tools and data sources. For example, a typical workflow for analyzing a CT scan might involve first running a segmentation model to identify regions of interest, then using a classification model to characterize those regions, and finally generating a structured report summarizing the findings. An agentic AI system can orchestrate this entire process autonomously, calling each tool in the correct sequence and synthesizing the results into a coherent output.

Three core properties characterize agentic systems: goal persistence (the system does not stop until the required deliverables are met), tool use (it can act on external systems, not just generate text), and iterative reasoning (each result informs the next decision). We evaluated several existing agentic frameworks, including LangChain, CrewAI, and AutoGen, but opted to build a custom agentic system. The primary reason was the need for tight integration with MCP: existing frameworks do not natively support MCP as a tool protocol, which would have required additional adapter layers and reintroduced the fragmentation problem we were trying to solve. Building a custom loop also allowed us to enforce specific behaviors required in a clinical context, such as a completion contract under which the agent cannot declare success while required deliverables remain unmet, and a debug mode in which the user must approve each tool call before execution.

Contrasting with simpler alternatives clarifies why this design was necessary. A plain chatbot produces one response per prompt and cannot maintain state or call tools. A hardcoded pipeline can call tools but relies on the developer to anticipate every possible execution path, making it brittle in the face of real-world variability. A standalone ML model provides predictions but has no language interface and cannot adapt its behavior based on context. We found that only a true agentic system could handle the complexity and variability of real clinical workflows, which often require conditional logic, error handling, and dynamic tool selection.

Our own system began as precisely this kind of hardcoded pipeline: in the initial version, the LLM was never consulted, and all sequencing decisions were made by explicit conditional logic in the orchestrator code. This approach proved limiting, as it could not adapt to new information without significant reprogramming, which motivated the transition to a fully agentic design.

This transition is not without its challenges. Reliable agentic behavior requires careful prompt engineering: the agent must be guided to select tools in the correct order, avoid premature declarations of success, and handle ambiguous or incomplete results gracefully. These challenges are discussed in detail in later sections. However, the fundamental advantage over any hardcoded alternative is scalability: supporting a new clinical domain does not require rewriting workflow logic but only adding a new MCP server that exposes the relevant tools, or a skill file that encodes the domain-specific protocol, and the agent discovers and incorporates both at runtime without code changes. This also opens a much wider architectural design space. Rather than a single fixed agent, one could configure specialist agents for distinct clinical tasks, use prompt-based behavioral switches to change the agent's operating mode, or compose multiple MCP servers covering entirely different domains. None of these extensions require touching the orchestration code, which is simply not true of any hardcoded pipeline.

When testing the system we explored two LLM backend approaches: a locally-hosted model and a cloud API. The local model was Ollama's deepseek-r1:7b, with llama3.1:8b also evaluated, both small enough to run on consumer hardware and allowing for complete data privacy since all processing happens locally. The API-based model was Gemini 2.0 Flash, with later versions used as token limits required, a significantly larger and more capable model hosted in the cloud. The local approach offers greater control and privacy but proved to struggle with the demands of multi-step clinical workflows. We observed consistent failures in tool calling: the model frequently failed to select the correct tool for the given context and could not reliably chain multiple calls to reach a goal. Addressing this consumed considerable development time across prompt engineering, workflow simplification, logging, and targeted adjustments. None of these fully resolved the problem, as the root cause was model size rather than configuration. Switching to the API-hosted model resolved all of these issues immediately. The API approach introduces trade-offs of its own, namely latency and potential data security concerns in privacy-sensitive deployments, but for the complex reasoning required by clinical imaging workflows it proved to be the only reliably viable option at the model sizes currently available for local inference. The system architecture supports both backends, allowing deployment to be tailored to the constraints of a given clinical environment.

C. MCP as a Standardized Tool Integration Layer

The conventional approach to agent-tool integration is the bespoke REST adapter: for each tool the agent must call, a custom connector is written that handles authentication, request serialization, response parsing, and error mapping. This approach does not scale. Each new tool adds a new adapter; changes to a tool API break the corresponding adapter; and

[2]An agentic AI system is one that pursues a goal through iterative reasoning and tool use rather than producing a single response. At each step the model decides what action to take next, executes it via an external tool or API, observes the result, and continues until the goal is reached or the iteration budget is exhausted.

the agent itself must encode assumptions about every tool it might call.

The Model Context Protocol (MCP) addresses this directly. MCP defines a client-server protocol in which each tool server declares its available tools at startup, specifying names, input schemas, and descriptions. The agent discovers tools dynamically at runtime and invokes them by name without requiring any prior knowledge of the server's internal implementation. This decouples the agent from individual tool APIs: adding a new tool server requires no changes to the agent, only a new MCP-compliant server that exposes the relevant capabilities.

Singh et al. (2025) survey MCP's architecture in detail, positioning it as the solution to fragmented API integrations for agentic AI systems [1]. The survey also identifies MCP sampling, a mechanism for LLM-to-LLM communication within the MCP protocol, as a path toward sub-agent specialization, in which a coordinating agent can delegate to specialist agents exposed as MCP servers. Singh et al. note that healthcare is a domain where these properties are especially relevant, but the survey discusses clinical applications only conceptually; no implemented system is presented.

D. Skills as Clinical Capability Packages

Tool access via MCP determines what an agent can do, it does not determine what the agent should do in a given clinical domain. A segmentation server, a terminology server, and a report template server can all be exposed as MCP tools, but the correct sequencing of those tools for a CT lung evaluation versus a pathology workflow requires domain knowledge that is separate from both the agent's general reasoning capability and the tool's own logic.

Skills[3] address this layer. Xu and Yan (2026) formalize agent skills as composable, on-demand capability packages that provide domain-specific behavioral guidance to an LLM without retraining it [2]. The key architectural insight is that skills and MCP are complementary rather than competing: skills define what to do in a given domain, while MCP exposes the tools with which to do it. Xu and Yan do not address healthcare-specific skill design, the governance question of who maintains skill files when clinical protocols change, or how skills compose with MCP in a working system, gaps that this work directly addresses.

E. Positioning of This Work

II. Methods

A. System Architecture Overview

MedGov-AI is organized into three layers: a React frontend, a FastAPI orchestrator, and a set of MCP tool servers. Each layer has a clearly defined responsibility, and the boundaries between them are designed so that clinical domain logic lives neither in the interface nor in the transport layer but entirely in the orchestrator, where it can be reasoned about, logged, and audited.

Frontend: The client layer is a React 18 application that provides the clinical user interface, communicating with the orchestrator exclusively via a REST and SSE API. SSE enables real-time streaming of task progress and agent responses to the user without polling.

Orchestrator: The orchestrator is a FastAPI application and is the system's single point of control: it exposes the REST and SSE API consumed by the frontend, owns all session and task state, manages connections to every MCP server, and hosts the three internal services that drive the system's behaviour: the `AgenticAgent`, the `task_runner`, and the `watcher_service`.

The `AgenticAgent` is the sole decision-maker in the system. A deliberate choice was made to use a single agent rather than a hierarchy of sub-agents, for two reasons: simplicity and auditability. A single agent is easier to reason about, debug, and extend, and every tool call, its inputs, and its result are recorded in a single session trace with no hidden delegation. The agent runs an iterative LLM loop using either the Gemini API or a locally-hosted Ollama model as its reasoning backend, and executes multi-turn function calling to invoke MCP tools in sequence. It maintains a session context across turns so that information produced by one tool call is available to the next without requiring the user to restate it.

The `task_runner` manages long-running operations that cannot complete within a synchronous HTTP request. Operations such as MONAI inference or Cellpose segmentation are submitted to a background thread pool and tracked through a state machine with four states: queued, running, completed, and failed. A semaphore limits concurrent GPU inference to a single job at a time, preventing out-of-memory crashes on constrained hardware. The task runner also captures stderr output from inference processes and, on failure, passes the log to the LLM for a plain-language error explanation before surfacing it to the user. Task state is persisted in SQLite so that progress survives backend restarts, and the frontend is notified of state transitions via SSE.

The `watcher_service` monitors registered filesystem directories using periodic polling and triggers the agent automatically when a new file is deposited. This enables event-driven clinical workflows in which files arriving from a PACS[4] system or a scanner initiate the full analysis pipeline without any user action. No changes to upstream systems are required; the watcher integrates at the filesystem level and streams its activity log to the frontend via a dedicated SSE[6] channel.

[3]In this context, a skill is a plain-text file that encodes domain-specific behavioral guidance for the agent - step-by-step workflow protocols, guard conditions, sequencing constraints, or bodies of domain knowledge such as regulatory standards or API usage patterns. Skills are not executable code; they are instructions that the agent reads and follows using its existing tools. The agent loads a skill on demand rather than at startup, keeping the system prompt minimal until a specific domain is required.

[4]Picture Archiving and Communication System (PACS): the institutional system used in clinical radiology to store, retrieve, and distribute medical images, typically in DICOM[5] format.

[6]Server-Sent Events (SSE): a unidirectional HTTP streaming mechanism through which the server pushes events to connected browser clients in real time, without the client needing to poll.

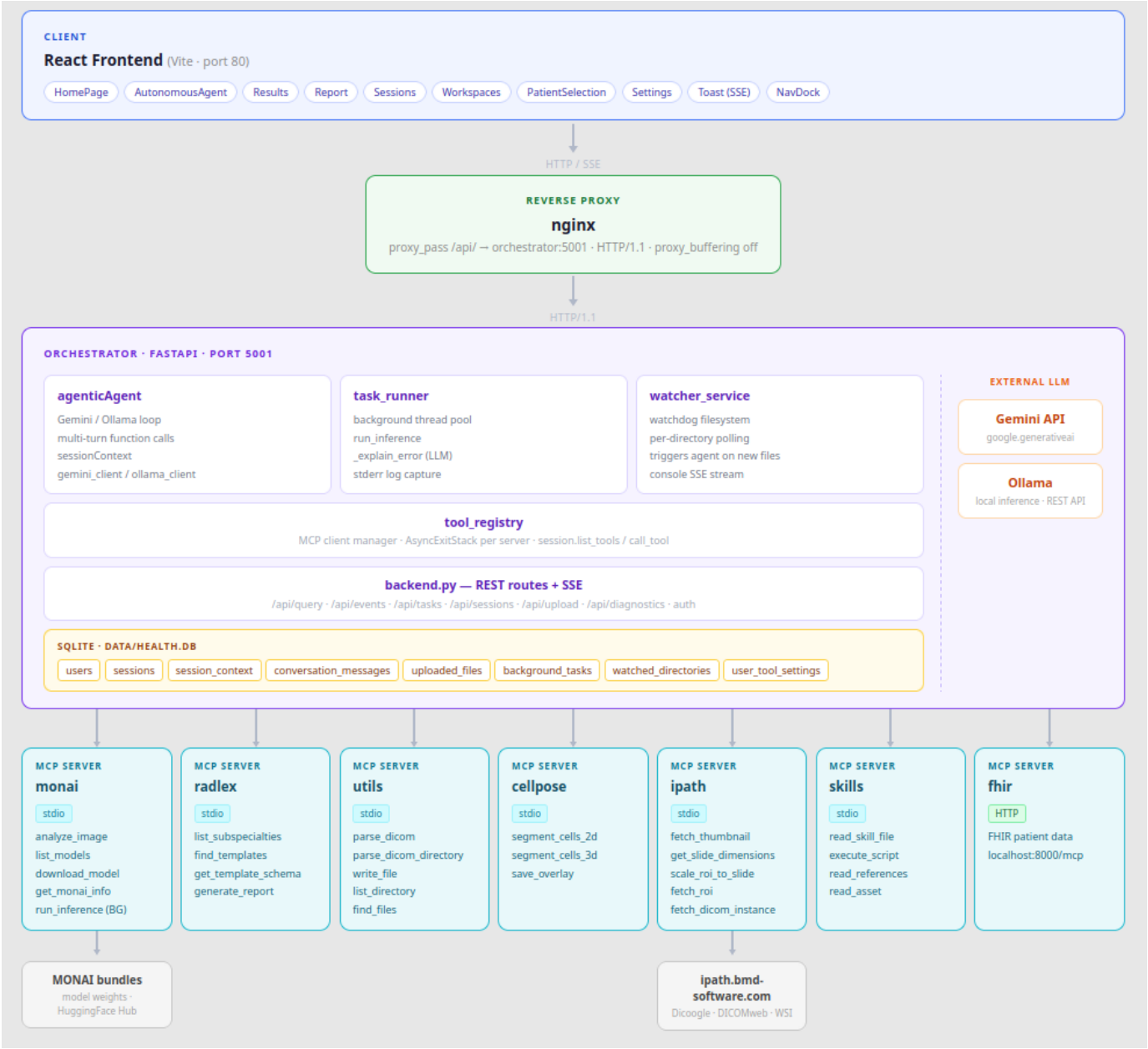

Fig. 1. MedGov-AI system architecture.

All persistent state is stored in a single SQLite database with WAL journaling. The schema covers users, sessions, conversation history, uploaded files, background task status, watched directories, and per-user tool settings.

LLM Backends: The orchestrator supports two LLM backends, selectable at deployment time without modifying any agent logic. The Gemini API provides a large cloud-hosted model capable of reliable multi-step function calling in complex clinical workflows, at the cost of external data transmission and API dependency. Ollama provides a local inference alternative using smaller open-weight models running entirely on the deployment host with no external data transmission, which is relevant in privacy-sensitive clinical environments, but proved insufficient for reliable tool calling at the 7–8B parameter scale currently available for local inference. Both backends implement the same internal interface.

MCP Server Layer: The tool layer consists of six MCP servers, each running as an independent process in its own isolated Python virtual environment. Isolation at the process level is a deliberate design choice: it prevents dependency conflicts between servers with incompatible requirements (most critically between the GPU-enabled PyTorch installation required by `mcp-monai` and the CPU-only installation used elsewhere, which cannot coexist in a single environment). Tool names are prefixed with the server name (e.g., `monai.run_inference`,

`radlex.generate_report`), providing unambiguous namespacing across servers. All servers communicate over stdio transport. The available servers and their clinical functions are listed in Table I.

This architecture means that extending the system with a new clinical capability requires only writing a new MCP-compliant server and registering it in the configuration file. The agent, the orchestrator, and every existing server remain unchanged. This is the property that makes MCP a viable interoperability layer rather than yet another bespoke integration approach.

B. MCP Server Layer

Each MCP server encapsulates a distinct clinical capability and exposes it as a set of named, schema-defined tools. The agent has no hardcoded knowledge of any server's internals. It discovers available tools at startup and selects among them based on context.

Mcp-monai is the medical image inference server, built on the MONAI framework [3] with PyTorch as the underlying deep learning backend. It detects the imaging modality and body part automatically from pixel statistics and file metadata, returning a ranked list of recommended models. The model catalogue can be filtered by modality, body part, or category, and bundles are retrieved from HuggingFace Hub and cached locally on demand. Inference uses a sliding window strategy to handle the variable dimensions of 3D medical volumes. Bundled models cover spleen segmentation on CT, pancreas and tumour segmentation on CT, lung nodule detection on CT, brain tumour segmentation on multi-channel MRI, whole-body CT segmentation across 104 anatomical structures, and kidney segmentation using the UNEsT architecture.

Mcp-radlex provides structured radiology reporting through integration with the RadLex terminology standard. The agent can browse available radiological subspecialties, search the template catalogue by subspecialty or keyword. When populating a report from raw inference results, the server uses MCP sampling to delegate the mapping of segmentation outputs to template fields back to the host LLM, which also generates the findings narrative, impression, and recommendations. This closes the loop between inference output and clinical documentation: the agent receives segmentation results, interprets them, and produces a report conforming to a standard radiological schema rather than free text.

Mcp-utils provides general-purpose filesystem and DICOM operations that underpin every clinical workflow. It extracts the full metadata of a DICOM file, including modality, body part, patient identifiers, acquisition parameters, and pixel data statistics, and can apply the same extraction across an entire directory. On the filesystem side, the server supports reading, writing, moving, copying, and deleting files, listing directory contents, and searching by pattern.

Mcp-cellpose provides cell segmentation for pathology workflows using the Cellpose v4 universal model (cpsam). Cellpose v3 exposed multiple task-specific models (cyto3, cyto2, nuclei, each approximately 80 MB) selected by a `model_type` argument. Cellpose v4 replaces all of these with a single SAM-based universal model (cpsam, approximately 2.5 GB) that was evaluated on both versions during development. Testing on the same pathology ROI image showed v3 (cyto3) detecting 1,100 cells against v4 (cpsam) detecting 1,703 cells, a 55% increase in detected count on identical input. v4 was selected as the deployment model. However, cpsam cannot distinguish cytoplasm from nuclei on standard RGB images without separate fluorescence channels, and the `model_type` argument accepted by the v3 API is silently ignored in v4. The server supports both 2D and 3D inputs and composites the predicted cell mask onto the original image for visual inspection. GPU availability is detected at startup with automatic CPU fallback, though CPU execution is only practical for small test images (see Section **??**).

Mcp-ipath integrates with the iPath telepathology platform, which aggregates whole slide images (WSI) and DICOM series via Dicoogle and DICOMweb. It supports thumbnail retrieval, resolution-level coordinate mapping, and region-of-interest extraction at arbitrary resolution levels without downloading the full WSI, as well as access to individual DICOM frames and series.

Mcp-skills exposes the skill system to the agent as callable tools, allowing it to load a skill's entry point and its associated reference documents and assets, and to execute predefined scripts associated with a skill. The skills system is described in detail in Section II-C.

C. Skills System

The skills system provides a mechanism for encoding domain-specific behavioral guidance that the agent loads on demand. A skill body can take different forms depending on the domain: a step-by-step workflow protocol with sequencing constraints and cross-server coordination, or a body of domain knowledge such as regulatory standards, clinical report templates, and API usage patterns. What these forms share is that the guidance cannot be derived from MCP tool schemas alone and is too domain-specific or voluminous to embed permanently in the system prompt. Embedding it permanently would increase token consumption on every request, including those that do not require that domain, and would increase the risk of the LLM overlooking or misapplying instructions buried in a large prompt, a well-documented failure mode in prompt engineering [4]. Skills address both problems by keeping the system prompt minimal and loading domain detail only when the agent determines it is needed for the current task.

1) Structure: A skill separates awareness from detail. Every available skill is surfaced to the agent at startup through a short name and natural language description kept permanently in the system prompt, while the full skill body remains unloaded until needed. When a request arrives, the agent reasons over these descriptions to identify the applicable skill, then loads its body on demand as operating instructions for that task. This enables progressive disclosure: the context window carries

TABLE I
MCP SERVERS AND CLINICAL FUNCTIONS

Server	Transport	Clinical Function
mcp-monai	stdio	Medical image segmentation and detection
mcp-radlex	stdio	RadLex terminology and report templates
mcp-utils	stdio	DICOM parsing and filesystem operations
mcp-skills	stdio	Skill workflow loading and execution
mcp-ipath	stdio	iPath telepathology integration
mcp-cellpose	stdio	2D/3D cell segmentation (Cellpose)

only a lightweight index at rest, expanding to include full domain detail only for the domain currently in use.

A skill body can itself be hierarchical, containing an entry-level document that routes to more specific sub-documents, reference material, templates, and scripts, each loaded independently as the task requires. This allows a single skill to cover a broad domain without forcing the agent to load everything at once.

2) Implemented Skills: Four skills are currently deployed in the system. The **dicom-analysis** skill encodes the tool sequence and guard conditions for DICOM analysis ending in a radiology report; the **wsi-tumor-detection** skill encodes the visual inspection and cell-counting workflow for whole-slide pathology images. The **clinical-reports** and **pydicom** skills are publicly available community skills [**?**]: clinical-reports encodes regulatory standards and structured templates for clinical documentation, and pydicom provides API guidance for reading, writing, anonymizing, and manipulating DICOM files programmatically. The imaging skills illustrate skills as workflow protocols; the community skills illustrate skills as domain knowledge packages, showing that the same mechanism covers both forms of guidance and that externally maintained skills can be adopted without modification.

3) Governance: A key property of this design is that skill files are plain text and can be updated without touching the agent, the MCP servers, or any other part of the codebase. When a workflow step changes (a different tool call order, a new guard condition, a revised output format) the relevant reference document is edited and the agent adopts the new behaviour immediately on the next session, with no software deployment required. This separates workflow maintenance from software development, which is relevant in clinical settings where the people who understand the correct procedure are not necessarily the people who maintain the codebase.

D. Agentic Execution Loop

The agent's operation is governed by an iterative execution loop that drives the system from a natural language goal to a set of completed deliverables, calling MCP tools at each step based on the LLM's reasoning about what to do next.

1) System Prompt and Session Initialisation: At the start of each session the agent receives a system prompt that establishes its role, its available tools, and its behavioural rules. The system prompt includes the list of available skills as a plain bulleted list of names and descriptions, injected at startup after dynamic discovery, so the agent knows what clinical domains it can operate in before any user message arrives. It also includes explicit rules for tool usage: when to read a skill file before acting, when to queue a long-running operation to the background task runner rather than calling it inline, how to handle DICOM series directories versus individual files, and how to avoid repeating failed tool calls. Two variants of the system prompt exist: one for autonomous mode, which omits confirmation-related guidance, and one for assisted mode, which includes it.

2) Iterative Reasoning and Tool Dispatch: Each iteration of the loop follows the same structure. The agent receives a prompt describing the current goal, any available files, and the results of all prior tool calls in the session. On the first iteration the prompt is minimal: goal, data, and file context. On subsequent iterations the prompt also includes the execution history, which the agent evaluates before deciding its next action. The LLM responds with either one or more function calls, or a text response indicating that the goal is complete or that more information is needed. Multiple tool calls can be returned in a single LLM response and are executed sequentially within the same iteration. All results are collected and returned to the LLM in a single message before the next iteration begins. Function calls are parsed and dispatched to the appropriate MCP server via the tool registry, and the result is appended to the execution history.

A repetition guard addresses a failure mode observed during development in which the agent would attempt the same failing tool call repeatedly rather than trying an alternative approach, consuming iterations without making progress. The guard inspects the execution history before each tool call: if the same tool has failed twice in a row, the third attempt is blocked and an error message is injected into the history instructing the agent to try a different tool or approach.

3) Built-in Tools: Alongside the MCP tools discovered at startup, the agent has access to a set of built-in tools that are handled directly by the orchestrator rather than by any MCP server. `goal_achieved` signals that the task is complete and carries a summary of the result; calling it is the only way for the agent to exit the loop successfully. `need_more_info` allows the agent to pause execution and ask the user a clarifying question when the goal is ambiguous or required inputs are missing. `queue_task` submits a long-running operation to the background task runner and returns immediately, allowing the agent to respond to the user without blocking on inference. `list_tasks` retrieves the current

status of queued or running background tasks, allowing the agent to check whether a previously submitted inference job has completed before proceeding. These tools give the agent explicit control over its own lifecycle and over the background task queue without exposing those concerns as MCP tool servers.

4) Operating Modes: The system supports two operating modes that can be selected per session. In **autonomous mode**, the agent executes tool calls without interruption from goal receipt to completion. This is the intended mode for event-driven workflows triggered by the workspace watcher, where no user is present to intervene. In **assisted mode** (referred to in the codebase as debug mode), the loop pauses before each tool call and returns a `confirmation_required` response to the frontend, which presents the proposed tool name and arguments to the user for approval or rejection before execution resumes.

E. Background Task Queue

F. Workspace Monitoring and Event-Driven Triggering

Not all clinical workflows are initiated by a user interacting with a chat interface. In many settings, imaging files are deposited into shared directories as part of existing institutional processes, with no operator present to trigger analysis manually. The workspace monitoring system addresses this by turning filesystem events into agent triggers, allowing MedGov-AI to process incoming files automatically without requiring modification to upstream workflows.

1) Architecture: Each workspace is a named, registered directory stored in the SQLite database alongside a configurable prompt that defines what the agent should do when a file arrives. At startup, the orchestrator resumes watching all previously registered workspaces. The `watcher_service` uses a polling observer to monitor each registered path recursively, bridging filesystem events into the orchestrator's asyncio event loop via an internal queue.

When a new file is detected, the service does not trigger the agent immediately. Instead it starts a three-second debounce window: if additional files arrive within that window, they are batched together. Only after three seconds of inactivity does the service dispatch all accumulated files to the agent in a single invocation. This prevents a burst of files, such as a DICOM series arriving slice by slice, from spawning multiple redundant agent executions.

2) Trigger Flow: When the debounce window closes, the watcher copies the incoming files from the watched directory into the corresponding workspace directory and constructs a goal prompt that includes the workspace name, the list of received files, and the preconfigured prompt for that workspace. It then invokes the agent in autonomous mode, since no user is present to confirm tool calls. The agent proceeds through its standard execution loop: reading the appropriate skill file (if needed), parsing the files, running the required analysis, and producing the configured outputs. The entire activity log, including file detection events, agent actions, and results, is streamed to the frontend via SSE under the workspace's dedicated console channel, giving an operator real-time visibility even when the workflow was triggered without user interaction.

G. Use Cases

Two clinical workflows were selected to evaluate the system. Together they cover the two imaging modalities the system targets, radiology and pathology, and exercise the full tool stack: DICOM parsing, segmentation inference, report generation, and whole slide image analysis. Both workflows can be initiated interactively through the chat interface or triggered automatically when files arrive in a monitored directory.

1) Radiology: DICOM Analysis and Report Generation: A clinician or automated trigger provides a DICOM file or series. The system identifies the study type, runs the appropriate segmentation model, and produces a structured preliminary radiology report matched to the modality and anatomy. Success is defined as a correctly identified study type, a completed segmentation, and a filled report template with findings mapped to the appropriate fields.

2) Pathology: Whole Slide Image Analysis: A whole slide image is provided. The system runs tumour detection and cell segmentation, returning segmentation results and a summary of the detected region. Success is defined as a completed segmentation run with quantitative output, specifically the detected region and cell count, returned without manual intervention.

H. Evaluation Setup

Evaluation was structured in three phases. Phase 1 evaluates both API-based and locally hosted language models for tool-calling reliability in clinical workflows. Phase 2 evaluates whether the agent can operate autonomously in clinical workflows. It first examines how skills provide the workflow constraints that make autonomous execution reliable, showing that the agent follows the correct tool sequence and guard conditions without human intervention. It then compares assisted and autonomous execution of the same task to assess whether human-in-the-loop confirmation changes the outcome, or whether the agent produces equivalent results in both modes. Phase 3 runs the two use cases end to end, comparing autonomous and manual workflows, evaluated by task completion and qualitative assessment of outputs, specifically whether the radiology report was correctly filled and whether segmentation completed successfully.

All experiments were run on a machine with an NVIDIA GeForce RTX 3050 6 GB GPU, using MONAI 1.5.1 and PyTorch 2.5.1 for inference. Input data consisted of publicly available DICOM datasets from the Cancer Imaging Archive [**?**] and the Medical Segmentation Decathlon [**?**]. Two categories of language model were tested: large API-based models (Gemini 2.0 Flash and Gemini 2.5 Flash) and locally hosted open-source models of significantly smaller scale (deepseek-r1:7b and llama3.1:8b via Ollama). [TODO: specify which datasets/tasks (e.g. which organ/modality from Decathlon, which collection from TCIA) — also add references.]

III. Results

A. Phase 1 - Model Experimentation

Four language models were evaluated: two large API-based models (Gemini 2.0 Flash and Gemini 2.5 Flash) and two locally-hosted open-source models via Ollama (deepseek-r1:7b and llama3.1:8b). The agent loop was capped at 10 iterations per task, meaning a model that cannot reach `goal_achieved` within 10 tool calls fails the task regardless of how many individual tool selections were correct.

1) Local Models: Both Ollama models were tested on a laptop with an RTX 3050 Ti GPU at two power profiles: 30W and 60W. At 30W, tool selection was generally correct on the first attempt, and in the majority of cases the first tool called was the appropriate one for the context. However, neither model reliably reached `goal_achieved` within the 10-iteration limit. The models would select correct individual tools but fail to chain them into a complete workflow, running out of iterations before producing a final result. At 60W, outcomes were mixed: some tasks completed successfully within the iteration budget, others did not. A consistent observation across both power profiles was that when a tool call failed, the local models rarely recovered by trying an alternative approach, confirming the need for the repetition guard described in Section II-C.

Each tool call with a locally-hosted model took approximately 20 seconds of inference time, making a 10-iteration task take up to three minutes of model time alone, before accounting for actual tool execution. This is a fundamental constraint of the model scale rather than a configuration issue.

A key finding from this phase is that two implementation choices significantly influenced local model reliability regardless of power profile: the quality of tool descriptions in each MCP server, and the availability of stateful execution history. Without clear, specific tool descriptions, the models defaulted to plausible-sounding but incorrect tool selections. Without execution history passed back at each iteration, the models repeated the same failing tool calls rather than adapting. Both of these are architectural properties of the system, not model-specific tunings, and both apply equally to the API-based backend.

2) API-Based Models: Gemini 2.0 Flash and Gemini 2.5 Flash resolved all of the reliability issues observed with local models. Both models completed multi-step clinical workflows within the iteration budget, correctly sequenced tool calls, and recovered from tool failures by selecting alternatives. Tool call latency was near-instant from the agent's perspective, reducing full task completion time by an order of magnitude compared to local inference. The primary limitations of the API approach are external: API availability is outside the system's control, and all data submitted to the agent is transmitted to a third-party service, which is a concern in privacy-sensitive clinical environments. A further practical consideration is token consumption. Because the agent passes its full execution history back to the LLM at each iteration, token usage grows with every tool call in the loop. Two concrete examples from development illustrate the scale of this problem. In an early version, image bytes were passed directly to the LLM instead of a file path, resulting in a single analysis call consuming approximately 60,000 tokens for a 57 MB scan. Separately, the `utils.parse_dicom_directory` tool was including all file paths in its response, adding approximately 27,000 tokens per call for large directories. After fixing both issues, a representative multi-file task begins at roughly 1,720 prompt tokens on the first iteration and grows to approximately 2,352 by the second, reflecting the expected accumulation rate from execution history.

Several measures were introduced to manage this growth. The agent uses Gemini's native stateful session mode, meaning only the latest tool result is transmitted per turn rather than rebuilding the full history in each prompt. Report generation uses MCP sampling, a lateral LLM call that produces the report narrative without adding to the agent's context window. The `get_template_schema` tool was removed after Henrique's refactor of the radlex server, eliminating a manual template inspection step that consumed tokens on every report workflow. The skills progressive-disclosure mechanism avoids embedding domain knowledge permanently in the system prompt by loading it only when the agent determines it is needed for the current task. These values and the measures taken represent the state of the system at time of writing; further optimizations remain possible.

3) Summary: The results confirm that the system architecture is backend-agnostic and functions with both local and API-hosted models; the gap lies in task completion reliability at current local model scales. Local models at 7–8B parameters can select appropriate tools but cannot reliably complete multi-step workflows within a bounded iteration budget. API-hosted models at significantly larger scale resolve this reliably. The architecture supports both backends without modification, allowing deployment to be matched to the constraints of a given environment: a privacy-critical deployment can use a local model with the understanding that task complexity must be managed, while a deployment where API access is available benefits from reliable autonomous operation. Table II summarises the observed differences.

B. Phase 2 - Autonomous Agent Execution

1) Skill-Guided Execution: This experiment evaluates whether loading a skill changes the agent's tool selection behaviour compared to operating without one, and whether the skill-prescribed sequence is actually followed in autonomous execution.

The **dicom-analysis** skill was used as the test case. Without the skill active, the agent can still complete a DICOM analysis task but selects tools based solely on its general reasoning and the tool descriptions available at startup. With the skill loaded, the agent receives an explicit step-by-step workflow: parse the DICOM header first, call `analyze_image` to confirm modality and body part, call `list_models` filtered to those parameters, download the model if not cached, and only then queue inference. Each step is a guard condition for the next:

TABLE II
LANGUAGE MODEL COMPARISON FOR AGENTIC TOOL-CALLING

Model	Type	Time/call	Task completion
deepseek-r1:7b (30W)	Local	≈20s	Failed within 10 iter.
llama3.1:8b (30W)	Local	≈20s	Failed within 10 iter.
Both models (60W)	Local	≈20s	Partial
Gemini 2.0 Flash	API	<1s	Consistent
Gemini 2.5 Flash	API	<1s	Consistent

inference is not attempted until the model is confirmed to match the scan.

In autonomous execution with the skill active, the agent followed this sequence without deviation across all test runs. Guard conditions were respected: on one occasion where `analyze_image` returned an ambiguous modality, the agent called `list_models` with both candidate modalities before selecting, rather than proceeding with the first result. Without the skill, the agent occasionally skipped the analysis step and called `list_models` directly with parameters inferred from the filename, which produced correct results in straightforward cases but would fail on files with non-descriptive names or atypical metadata.

A comparable observation was made in workspace monitoring execution. A deployment trace from March 2026 shows the agent autonomously handling an unrecognised file: it called `utils.move_file`, attempted `monai.analyze_image` (which returned a DICOM header error), created an `uncertain/` subdirectory, moved the file there, and called `goal_achieved`, completing the task in six iterations without any user input. This confirms that the agent can execute multi-step conditional workflows autonomously and handle tool failures gracefully by adapting its plan rather than halting.

The conclusion from this phase is that skills are not required for the agent to complete tasks, but they are what makes autonomous execution *reliable*: they constrain the agent to the correct domain-specific sequence and prevent it from taking plausible but clinically incorrect shortcuts.

A complementary observation concerns the relative efficiency of MCP tools versus skills for the same task. When a direct MCP tool exists for an operation, the agent consistently prefers it over the equivalent skill-based approach. This is expected: the agent reasons over tool descriptions at each iteration and a single tool call is always cheaper than the three-to-four calls required to load a skill (read entry point, optionally read references, execute). A concrete example from development: extracting DICOM metadata via the `utils.parse_dicom` MCP tool requires one call, whereas the `pydicom` skill achieves the same result through `skills.read_skill_file`, `skills.read_references`, and `skills.execute_script` - three calls and additional context window usage. Skills are therefore most valuable where no direct MCP tool exists, where workflow sequencing across multiple tools must be constrained, or where the domain knowledge to be loaded is too large or too domain-specific to embed permanently in the system prompt.

2) Assisted vs Autonomous Mode: The same DICOM analysis task was executed twice: once in assisted mode, where the agent paused before each tool call and presented the proposed tool name and arguments to the user for confirmation, and once in autonomous mode, where all tool calls were executed without interruption. The goal prompt, uploaded files, and active skill were identical in both runs.

The tool call sequences were identical across both executions. The agent selected the same tools in the same order, passed the same arguments, and reached `goal_achieved` in the same number of iterations. No tool call was rejected or modified by the user during the assisted run, and the outputs produced, segmentation results and per-organ volumes in the Results tab, were equivalent.

This result has a direct architectural implication. If assisted mode were a correction mechanism, the two sequences would diverge whenever the user overrode a proposed tool call. The fact that they did not indicates that the agent's autonomous decisions were already correct, and that the confirmation step added visibility without changing the outcome. Assisted mode is therefore best understood as a transparency and audit mechanism for operators who want to inspect each tool call before it executes, rather than a safety net that catches errors the agent would otherwise make. This in turn validates autonomous mode for event-driven workflows such as Use Case 2, where no user is present: the agent produces the same result whether or not a human is watching.

C. Phase 3 - Use Case Comparison

1) Use Case 1 - Interactive DICOM Analysis and Segmentation: **Clinical context.** A clinician has a DICOM scan and needs segmentation without knowing which model to use or how to chain the required tools, or simply wants to accelerate their work by dropping the DICOM files and looking directly at the processed and interpreted data.

Setup. The user uploads either a single DICOM volume file or a directory of DICOM slices forming a 3D series through the frontend file selector, and enters the following prompt in the chat to initiate the workflow:

> *"Analyze this CT scan and run the appropriate segmentation model."*

Workflow. The workflow is illustrated in Figure 2. The agent calls `parse_dicom_file` to extract modality, body part, and acquisition metadata from the DICOM header, then `analyze_image` to determine image type, intensity, and

spatial dimensions. Based on these findings, `list_models` returns a filtered list of compatible segmentation models. If the selected model is not cached locally, `download_model` retrieves it from HuggingFace Hub. Inference is then submitted to the background task queue via `run_inference`, which applies a sliding window strategy to process the 3D volume without exceeding GPU memory limits. The agent responds immediately after queuing without waiting for inference to complete. When the background task finishes, an SSE notification is pushed to the frontend and the segmentation results, including per-organ volumes, are displayed in the Results tab.

2) Use Case 2 - Autonomous Pathology ROI Monitoring: **Clinical context.** A pathology instrument or PACS system exports region-of-interest (ROI) images to a local directory as part of its standard output. No human is available to initiate analysis for each arriving file. The goal is for the system to detect each new file, run cell segmentation automatically, and route the result based on a configurable cell count threshold, without any user interaction. Files can also be deposited manually through the Workspaces tab in the frontend, which provides a console showing detection events and agent activity in real time.

Setup. A workspace is registered pointing to the watched directory. The following prompt is configured for that workspace, defining the agent's behaviour for every arriving file:

> *"When a new file arrives, check whether it is a pathology ROI image. If it is, run Cellpose cell segmentation and count the cells. If the cell count exceeds 2000, copy the segmentation mask to the output folder and log the result. If the file is not a valid ROI image, move it to the uncertain folder."*

Workflow. The workflow is illustrated in Figure 3. When the polling observer detects a new file, the watcher copies it to the workspace incoming folder and triggers the agent after a three-second debounce window. The agent calls `get_file_metadata` to determine file type, extension, and dimensions, and attempts to classify whether the file is a pathology ROI image. This classification step is an inherent limitation of the current design: reliably determining whether an arbitrary image is a valid ROI would require running a dedicated detection model, effectively doubling the computational cost before any analysis begins. To avoid this overhead, the workspace prompt instructs the agent to make a best-effort classification from file metadata and dimensions alone, and to route uncertain cases to a dedicated folder for manual review rather than proceeding with segmentation. This trades false-negative robustness for efficiency, and is an acknowledged design constraint rather than a failure of the workflow. If the file is classified as a valid ROI image, `segment_cells_2d` is submitted to the background task queue using the Cellpose v4 universal model (cpsam). When the task completes, the result is pushed to the frontend via SSE and displayed in the Workspaces console. If the cell count exceeds 2000, the segmentation mask is copied to the output folder; otherwise only a JSON log entry is written. Final results, including cell count, mask path, and timestamp, are persisted and accessible in the Results tab.

3) Inference Hardware Requirements: Inference performance is tightly coupled to available hardware, and the gap between CPU and GPU execution has direct clinical implications.

The Cellpose v4 model (cpsam) is a SAM-based architecture designed for GPU execution. On CPU, inference took 95 seconds on a 64×64 pixel test image; a real pathology ROI image at typical resolution caused the process to run for approximately three hours before hanging. GPU execution is estimated to be more than ten times faster based on manufacturer documentation and community benchmarks [**?**]. This makes GPU availability a practical prerequisite for the pathology use case rather than an optional accelerator. The MONAI inference pipeline is similarly GPU-dependent: on the test machine (RTX 3050 6 GB), certain models exceeded available VRAM and caused the subprocess to terminate without producing any output or error message. This failure mode was only diagnosed after adding stderr capture to the task runner, which then surfaced the underlying PyTorch out-of-memory exception. These findings shaped deployment requirements: the system is shipped with a GPU-enabled Docker configuration as the recommended deployment target, with CPU fallback available for development and testing only.

A practical advantage of both workflows, independent of accuracy, is the convenience the system provides. Processing speed depends on available hardware, cached models, and firmware, and is therefore highly deployment-specific. What the architecture guarantees regardless of hardware is that long-running operations are queued automatically, results are persisted, and the clinician can return to completed outputs without having been present during execution. In the workspace monitoring scenario this translates directly to overnight or unattended processing: files deposited at the end of a working day are analyzed and results are available the next morning. Meaningful evaluation of this property at scale, however, requires a broader set of test datasets and input types covering diverse modalities, resolutions, and acquisition protocols, which represents the primary direction for further experimental work.

IV. Discussion

V. Conclusions

References

[1] A. Singh, A. Ehtesham, S. Kumar, and T. T. Khoei, "A survey of the model context protocol (mcp): Standardizing context to enhance large language models (llms)," *Preprints*, April 2025. [Online]. Available: https://doi.org/10.20944/preprints202504.0245.v1

[2] R. Xu and Y. Yan, "Agent skills for large language models: Architecture, acquisition, security, and the path forward," 2026. [Online]. Available: https://arxiv.org/abs/2602.12430

[3] T. M. Consortium, "Project monai," Dec. 2020. [Online]. Available: https://doi.org/10.5281/zenodo.4323059

[4] D. Jaroslawicz, B. Whiting, P. Shah, and K. Maamari, "How many instructions can LLMs follow at once?" *arXiv preprint arXiv:2507.11538*, 2025. [Online]. Available: https://arxiv.org/abs/2507.11538

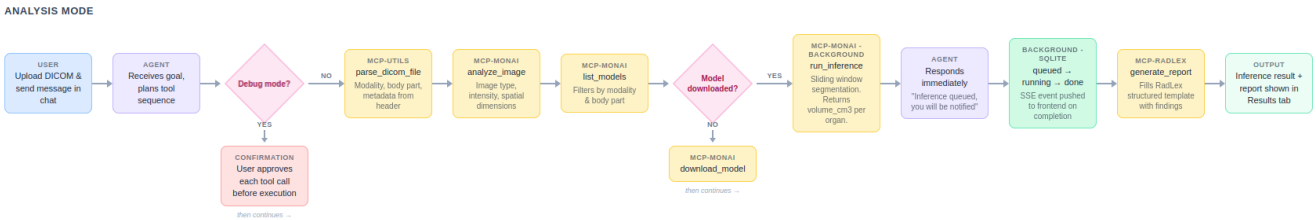

Fig. 2. Use Case 1 - Interactive DICOM analysis and segmentation workflow.

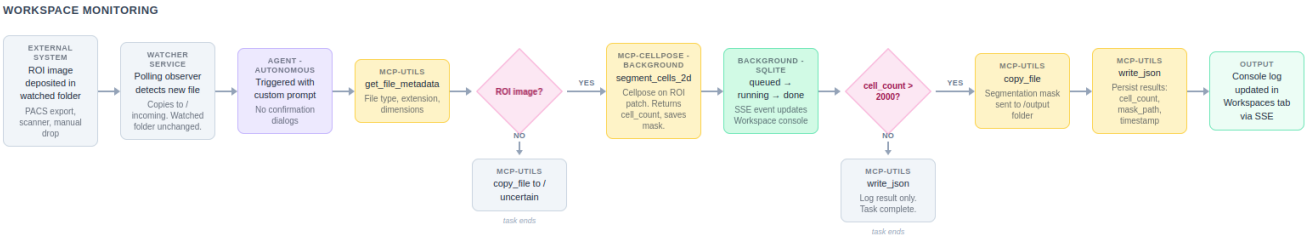

Fig. 3. Use Case 2 - Autonomous pathology ROI monitoring workflow.